Subject: Advanced Algorithms (ST5003CEM)

Core Modules:

1. **Complexity and Algorithms for NP-Hard problems:** Review of Big O-notation and algorithmic efficiency, Heuristics, Exhaustive search, Local search, GRASP, Simulated Annealing.
2. **Data Structures:** Heaps, Binary Search Trees, Self-Balancing Trees, Graphs and Hash Tables.
3. **Algorithms:** Divide and Conquer strategies, Greedy algorithms, Backtracking, Dynamic Programming and specific algorithms such as Prim's algorithm, Dijkstra's algorithm, Bellman-Ford algorithm
4. **Multi-threaded applications:Programming in Posix Threads**: Data sharing in Multi-threaded environments, data consistency and program stability.

Chapter 1:

Complexity and Algorithms for NP Hard Problems

Topics we study:

1. P
2. NP
3. NP-Complete
4. NP-Hard

Chapter 1: Complexity and Algorithms for NP Hard problems

Historical Evolution of Complexity Theory

Early Algorithm Analysis (1930s–1960s)

The foundation of algorithm complexity began with the concept of computation.

Major milestones

Year	Scientist	Contribution
1936	Alan Turing	Introduced the **Turing Machine**, a theoretical model of computation
1940	John von Neumann	Formalized stored-program computers
1965	Jack Edmonds	Introduced idea of **polynomial time algorithms**

source[1]

Chapter 1: Complexity and Algorithms for NP Hard problems

What is P class in computational complexity?

source[1][2]

1. Also known as PTIME or DTIME is a fundamental complexity class.
2. It is a set of class of decision problem which is solved in Polynomial Time with deterministic algorithm.
3. A smart algorithm that works quickly and efficiently, regardless of how large the problem gets.
4. Cobham and Edmonds are generally credited with the primary invention of the formal concept in 1965.
5. Computation problems that are efficiently “solvable” or “tractable”.

Chapter 1: Complexity and Algorithms for NP Hard problems

What is deterministic algorithm?

source[1][2]

1. What "Deterministic" actually means:
   An algorithm is deterministic if, given a specific input, it always follows the exact same path and produces the same output every time. There is no "branching" or "guessing." Each step is uniquely determined by the previous step.

2. The "Polynomial Time" part (P Class):
   If a deterministic algorithm can solve a problem in Polynomial Time, we say that problem belongs to the P class.It doesn't matter if the problem is "easy" or "hard" in terms of logic; if a standard computer can solve it efficiently even in the worst-case scenario, it is in P.

Chapter 1: Complexity and Algorithms for NP Hard problems

What is P?

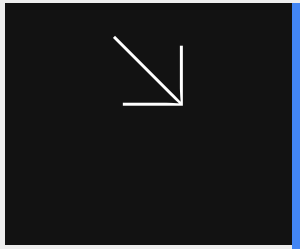
Examples:

1. Binary search: O(log n)
2. Merge sort: O(nlog n)
3. Shortest path (Dijkstra):(O(V+E)LogV)

Chapter 1: Complexity and Algorithms for NP Hard problems

What is NP?

source[7][8][9]



1. It is a set of class of decision problem which is solved in Polynomial Time with Non deterministic algorithm.
2. NP stands for Non-deterministic Polynomial time.
3. A problem is in NP if its solution can be verified in polynomial time.
4. These problems are hard to solve, but easy to verify.

Chapter 1: Complexity and Algorithms for NP Hard problems

Key Characteristics of NP?

source[8][9]

The class NP can be defined in two equivalent ways:

1. Solvable Nondeterministically: A problem is in NP if a nondeterministic Turing machine can solve it in polynomial time. In this model, the machine "guesses" a potential solution and verifies it instantly.

2. Verifiable Deterministically: A problem is in NP if, given a "yes" instance and a piece of evidence (called a certificate or witness), a standard computer can verify the correctness of that evidence in polynomial time

Chapter 1: Complexity and Algorithms for NP Hard problems

What is Nondeterministic?

In the theoretical world of NP, a nondeterministic algorithm doesn't use "guidance" or "heuristics" like a human would.
Instead, it follows a "Guess and Check" framework:

1. **The Guessing Phase:** The machine effectively "splits" into millions of copies of itself. Each copy picks one possible solution (a specific path, a set of numbers, etc.). This isn't one machine guessing randomly; it's a massive "branching" where every possible guess is made simultaneously.

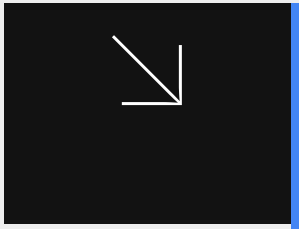
2. **The Verification Phase:** Each branch then checks its specific guess using a standard, logical set of rules.

3. **The Result:** If any of those branches find a "Yes," the whole algorithm returns "Yes" immediately.

source[7][8][9][10]

Chapter 1: Complexity and Algorithms for NP Hard problems

Example



Question : What's a problem where verifying a solution is easier than finding one?

Example: Sudoku problem. Solving Sudoku from scratch is hard.

Note:Checking a completed Sudoku grid is easy (polynomial time), but solving it may take exponential search.

Chapter 1: Complexity and Algorithms for NP Hard problems

What is NP-Complete?

- It is a class of computational problem if it satisfies two condition:
  a. It must be in NP class:a solution can be verified quickly.
  b. Another one is it must be as hard as every other NP problem:Every NP problem can be converted (reduced) into this problem.If we could devise a polynomial-time algorithm to efficiently solve any single NP-Complete problem, it would revolutionize the world of computation.
- First NP-Complete problem was proven by Stephen Cook in 1971. Problem is a Boolean Satisfiability Problem(SAT). This is called Cook's Theorem. Every NP problem can be transformed into SAT in polynomial time.
- Later Richard Karp showed 21 important problems are NP-Complete.
- Widely used in Cryptography and optimization in AI.

source[1][2]

Chapter 1: Complexity and Algorithms for NP Hard problems

Question : **Is there a problem that's so tough within NP that solving it efficiently could revolutionize problem-solving across the board?**

Important Note: If one NP-Complete problem is solved in polynomial time then:

we can say P = NP, meaning all NP problems become easy.

This is the one biggest unsolved problems in the computer science.

Source:[1]

Chapter 1: Complexity and Algorithms for NP Hard problems

What is NP-Hard?

1. NP-hard (Non-deterministic Polynomial-time hard) problems are a class of computational problems that are at least as difficult as the hardest problems in NP. They are characterized by their extreme computational complexity, meaning no efficient (polynomial-time) algorithm is known to solve them, and they are typically used for optimization tasks.
2. While NP-Hard match NP-Complete problems in their level of intricacy, they may not necessarily reside within the NP category themselves.
3. NP-Hard do not have the same convenient verification property that NP problems do.
4. This absence of easy verification sets them apart, making them formidable adversaries for problem solvers and algorithm designers.
5. Solution: Tackling NP-Hard problems requires ingenuity, clever algorithm design, and, at times, heuristic approaches.

Source:[1][3]

Examples of NP-Complete and NP-Hard problems

1. **Traveling Salesman Problem**
   a. Decision version (Is there a tour ≤ k distance?) → NP-Complete
   b. Optimization version (Find the shortest tour) → NP-Hard
2. **Graph Coloring Problem**
   a. Decision version (Can the graph be colored with k colors?) → NP-Complete
   b. Optimization version (Find the minimum number of colors) → NP-Hard

Source:[1]

Chapter 1: Complexity and Algorithms for NP Hard problems

What is Decision Problem?

- A **decision problem** asks a **YES or NO question**.
- Is there a solution that satisfies a condition? These problems belong to NP because the answer can be verified quickly.
- Decision problems ask "Does a solution exist?"

What is Optimization Problem?

- An **optimization problem** asks for the **best possible solution**.
- Optimization problems ask "What is the best solution?"
- Find the minimum or maximum value.
- Examples:
  - minimum cost
  - shortest route
  - maximum profit

Chapter 1: Complexity and Algorithms for NP Hard problems

Example 1: Traveling Salesman (Decision Version)

Problem:

Cities: A, B, C, D

Question:

Is there a tour visiting all cities with total distance ≤ 20 km?

Answer must be:

YES or NO

If someone gives a route:

A → B → C → D → A

You just calculate the distance and verify.

This is why the decision version is NP-Complete.

Example 2: Traveling Salesman (Optimization Version)

Problem:

Find the shortest possible route that visits all cities.

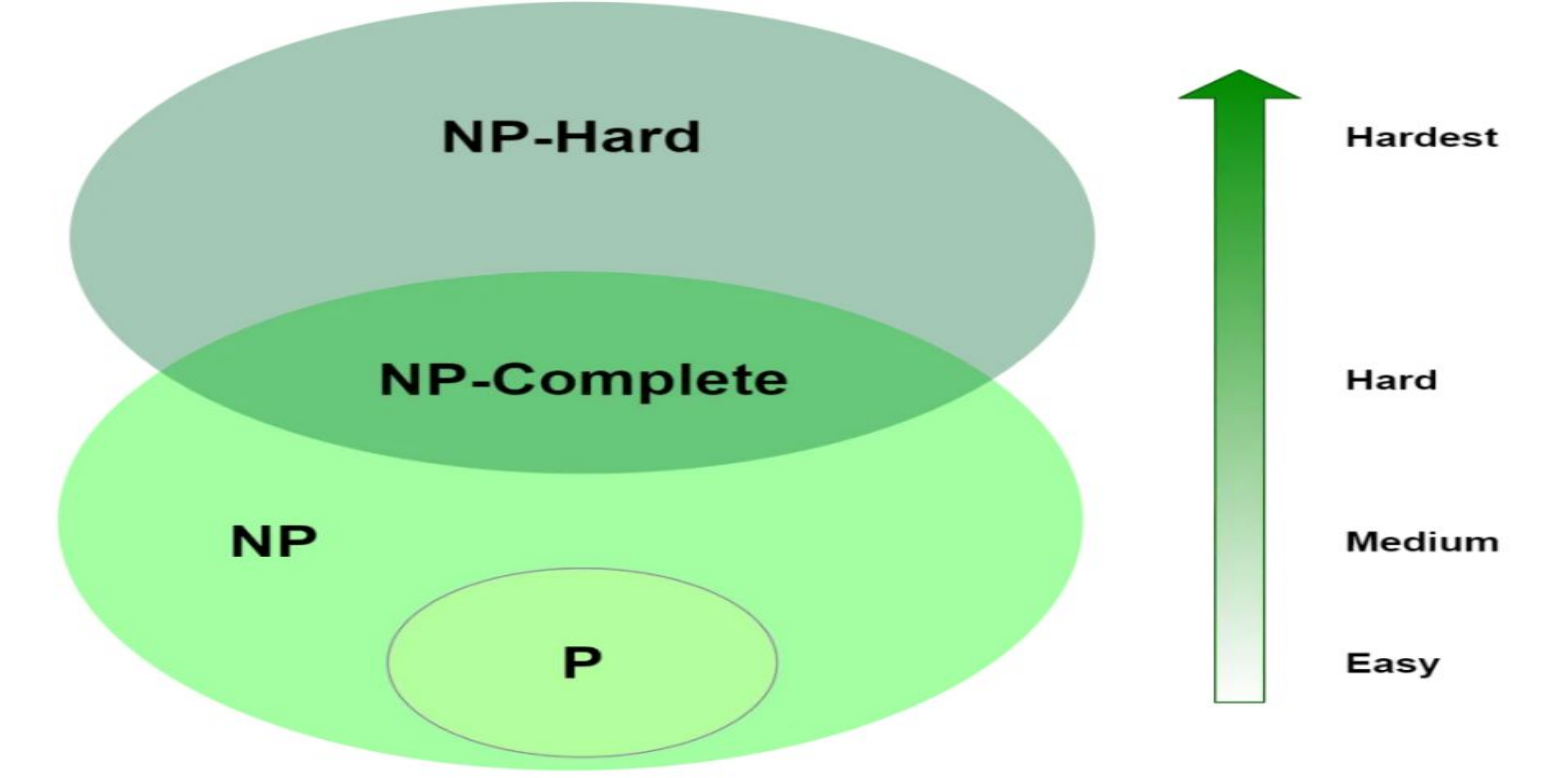
Output example:

Best route = A → C → B → D → A
Distance = 18 km

This requires searching many possibilities.

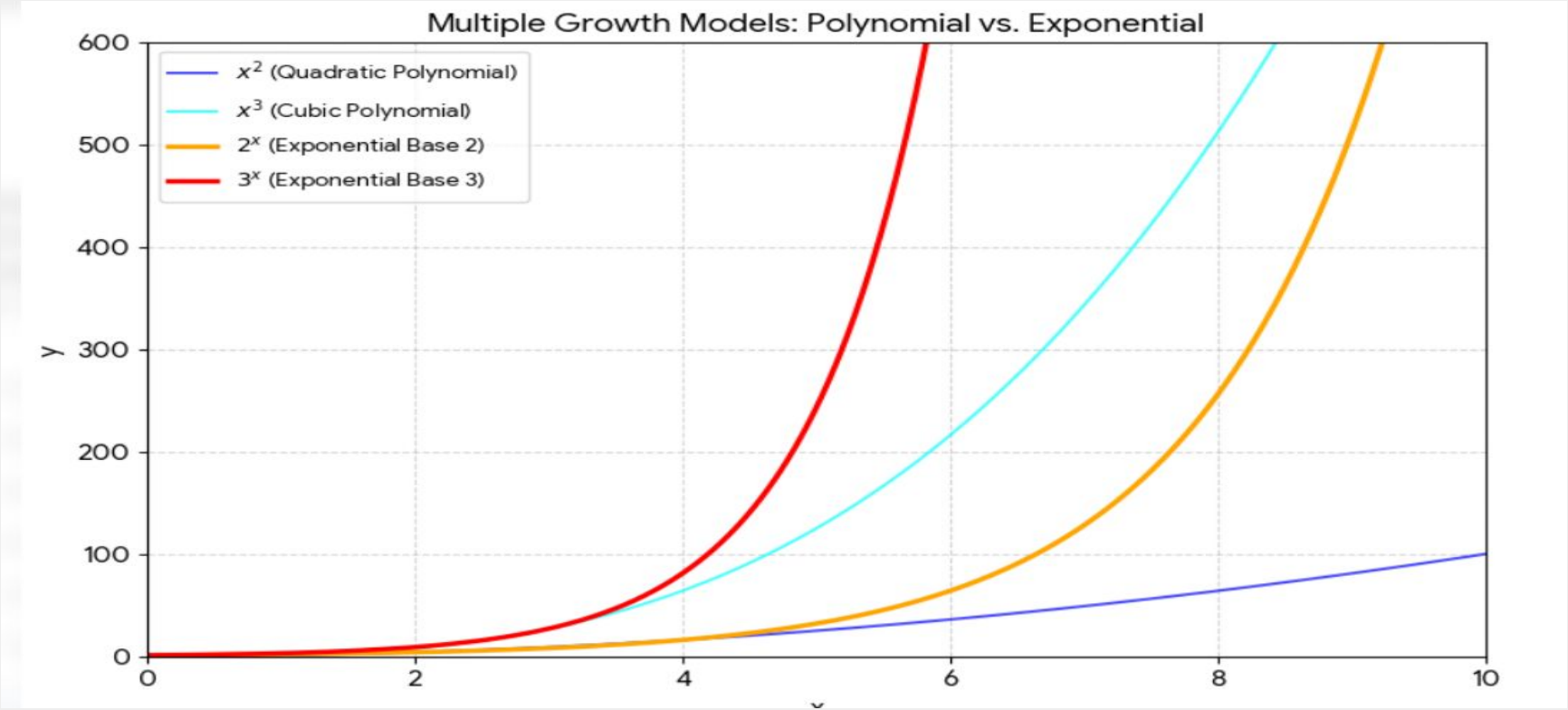
This version is **NP-Hard**.

Chapter 1: Complexity and Algorithms for NP Hard problems



Source:[1]

Exponential vs Polynomial



Source:[1]

Since optimal solution are unknown so we apply:

1. Approximation Algorithm
2. Heuristic Algorithm
    a. Local Search
    b. Metaheuristic
        i. Simulated Annealing
        ii. GRASP(Greedy Randomized Adaptive Search Procedure)
3. Exhaustive Search

Chapter 1: Complexity and Algorithms for NP Hard problems

Approximation Algorithm

1. An approximation algorithm is a technique used to find near-optimal solutions for complex optimization problems, particularly for NP-hard problems, in polynomial time.
2. The design and analysis of approximation algorithms include a mathematical proof confirming the quality of the generated solutions in the worst case, distinguishing them from heuristic approaches, which find reasonably good solutions, but do not provide any clear indication about the quality of the solutions.
3. Some examples are: **Travelling Salesman Problem, The Subset Sum Problem**

Source:[5][6][7]

Chapter 1: Complexity and Algorithms for NP Hard problems

Heuristic Algorithm vs Approximation Algorithm

1. **Performance Guarantees:** Approximation algorithms come with a theoretical guarantee (e.g., "solution is within 50% of optimal"). Heuristics offer no such guarantee and might produce very poor solutions in specific cases.
2. **Optimal Solution Proximity:** Approximation algorithms ensure the solution is within a certain factor of the optimum, whereas heuristics just aim for a "good enough" solution.

Source:[5][6][7]

Chapter 1: Complexity and Algorithms for NP Hard problems

What is Heuristic Algorithm?

Definition:

A heuristic algorithm is defined as a problem-solving method designed for specific problems, utilizing experience to exploit problem characteristics and generate high-quality f**easible solutions within reasonable computational time**. While these solutions **are not guaranteed to be optimal**, they serve as effective shortcuts when time is a critical factor.

Notes:

1. Heuristic algorithms are widely used in combinatorial optimization, graph theory, complexity theory of algorithms, and artificial intelligence, among other domains.

2. These algorithms do not guarantee exact solutions but aim to return the best and most acceptable answers according to the problem requirements, often trading off optimality for speed and flexibility.

3. It is a **Informed Search.**

Source:[11]

Chapter 1: Complexity and Algorithms for NP Hard problems

Heuristic Search and Heuristic Function

1. Heuristic Search:
   a. It **tries to optimize** the problem using heuristic function
2. Heuristic Function:
   a. It is a function that gives an estimation of the path from **Start Node or State to Goal State or Node.**

Types of Heuristic Function

1. **Admissible (h(n)<= h'(n))**
2. **Non Admissible (h(n)> h'(n))**

Real World Examples:

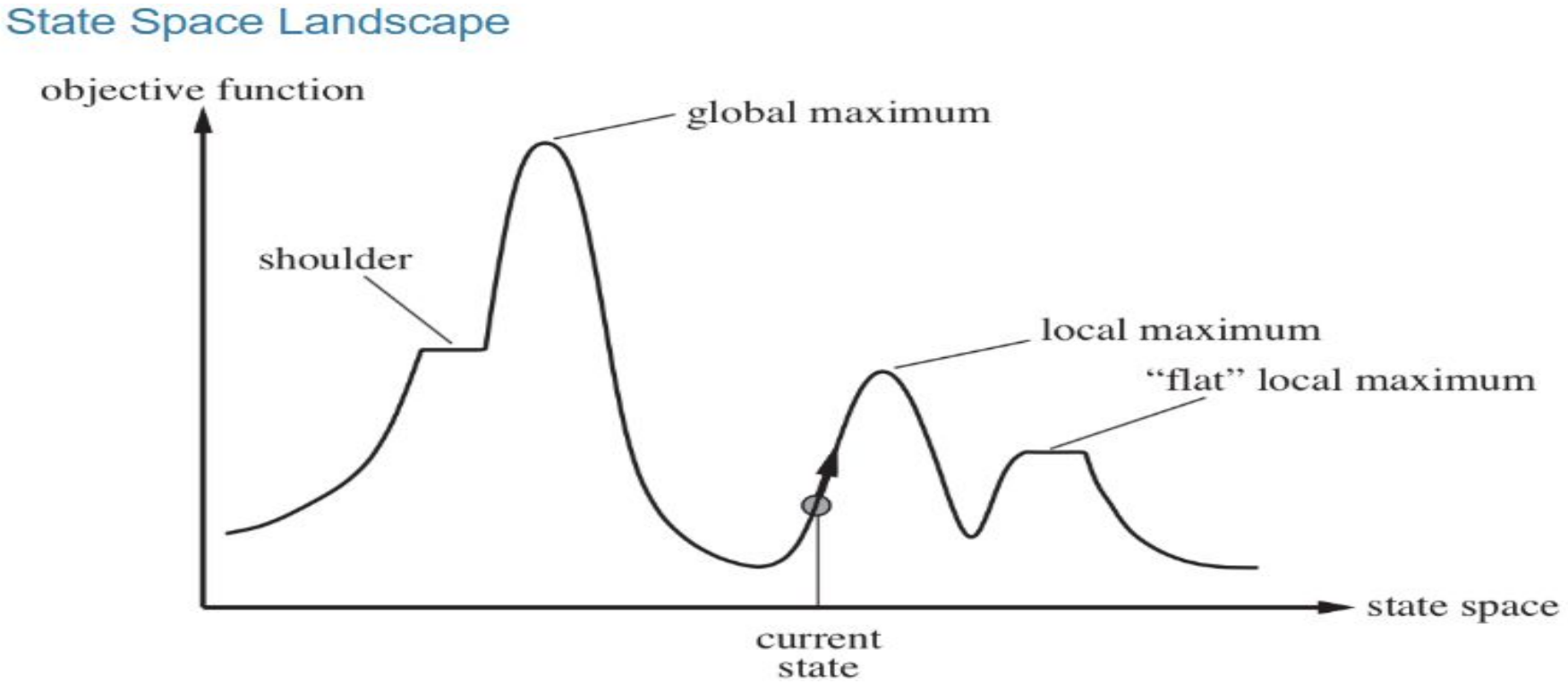
Logistics and transportation (vehicle routing), AI search (pathfinding in games), and daily human decision-making (using mental shortcuts or "rules of thumb" to save time).

Source:[12][13]

Chapter 1: Complexity and Algorithms for NP Hard problems

1. **Problems we solve:**
   a. **8 Puzzle**
   b. **Travelling Salesman**
   c. **Subset Sum**
   d. **Knapsack problem**

What is Local Search Algorithm?



Source:[15]

Chapter 1: Complexity and Algorithms for NP Hard problems

What is Local Search Algorithm?

Definition:

In computer science, **local search is a heuristic method** for solving computationally hard optimization problems. A **simple hill-climbing process** that improves a solution by moving to better neighboring solutions until no further improvement is possible. It gets stuck in local optima.

Notes:

1. Local search algorithms **move from solution to solution in the space of candidate solutions** (the search space) by applying local changes, until a solution deemed optimal is found or a time bound is elapsed.
2. The key idea behind a local search algorithm is to **start at an initial search position**—typically a randomly chosen(initial guess), complete variable assignment—and, at each step, move to a position selected from the local neighborhood, often based on a **heuristic evaluation function**. **They can quickly find good answers, especially when finding the perfect solution would take too long or too much effort.**
3. This process is **iterated until a termination criterion** is satisfied, such as finding a solution or reaching a local optimum.

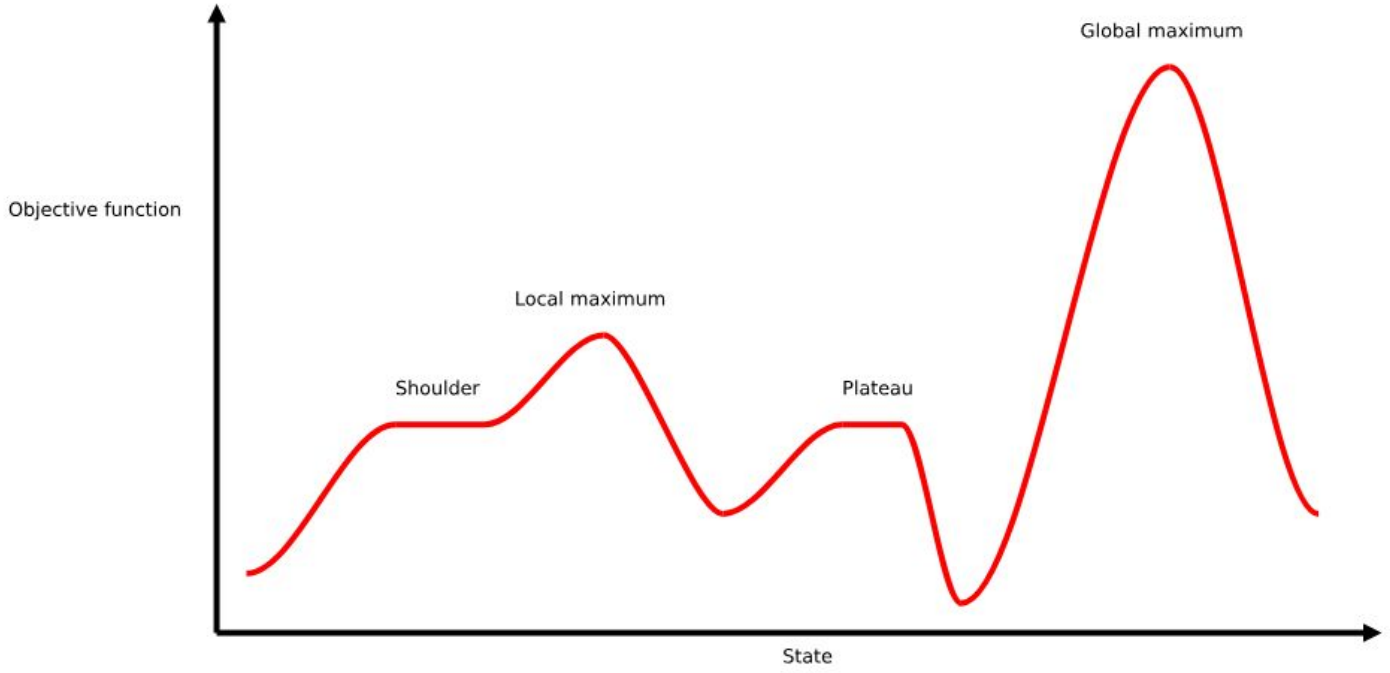
Source:[14][16]

Types of Local Search Algorithms

1. Hill Climbing
2. Simulated Annealing
3. A*

Source:[14][16]

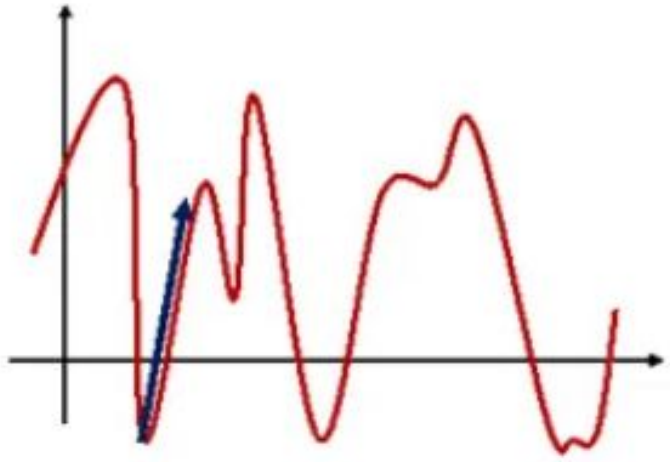
Hill Climbing method



Source:[30]

Hill Climbing method

1. Hill climbing is one of the **simplest optimization algorithms** to understand the program. **It's like following a basic rule: "If something's better, go there." This makes it a great starting point for solving many problems.**
2. When the problem is straightforward, **hill climbing can find good solutions quickly**. It doesn't waste time exploring every possible solution—it just follows the path upward.
3. The algorithm **doesn't need much computer memory or processing power**. It only needs to remember where it is and look at nearby solutions, making it practical for many real-world problems.
4. Hill climbing is **also called greedy local search with no backtracking**.



Hill Climbing Algorithm

Source:[18]

Hill Climbing method: How it works?

1. Start: Begin at a random point on the problem landscape.
2. Evaluate: Assess the current position.
3. Look Around: Check neighboring point(new solution and evaluate them.
4. Can Improve: Determine if any neighbors are better (higher value) than the current position.
5. Move: If a better neighbor exist, move to that point.
6. Repeat: Continue the process form the new point.
7. No improvement: If no better neighbors are found, you have reached a peak.
8. End: The process stops when no further improvement can be made.

Source:[18]

Hill Climbing method: Limitations

1. Getting stuck on small hills(Local optima)
2. The flat ground problem
3. The ridge problem
4. Starting point matters a lot

Source:[18]

Strategies to overcome Hill Climbing Limitations

1. Random-restart hill climbing
2. Simulated annealing

Source:[18]

Types of Hill Climbing method

1. Simple Hill Climbing
2. Steepest-ascent hill climbing
3. Stochastic Hill Climbing

Source:[18]

Hill Climbing method: Applications

1. Machine learning model optimization
    a. Feature Selection
    b. Hyperparameter tuning
    c. Neural Network training
2. Robotics and path planning
    a. Motion planning
    b. Joint angle optimization
3. Natural Language Processing
    a. Text summarization
    b. Word embedding

Source:[18]

Simulated Annealing

Source:[26][27]

Simulated Annealing

Figure 1: Heating the metal

Figure 2: Cooling the heated metal

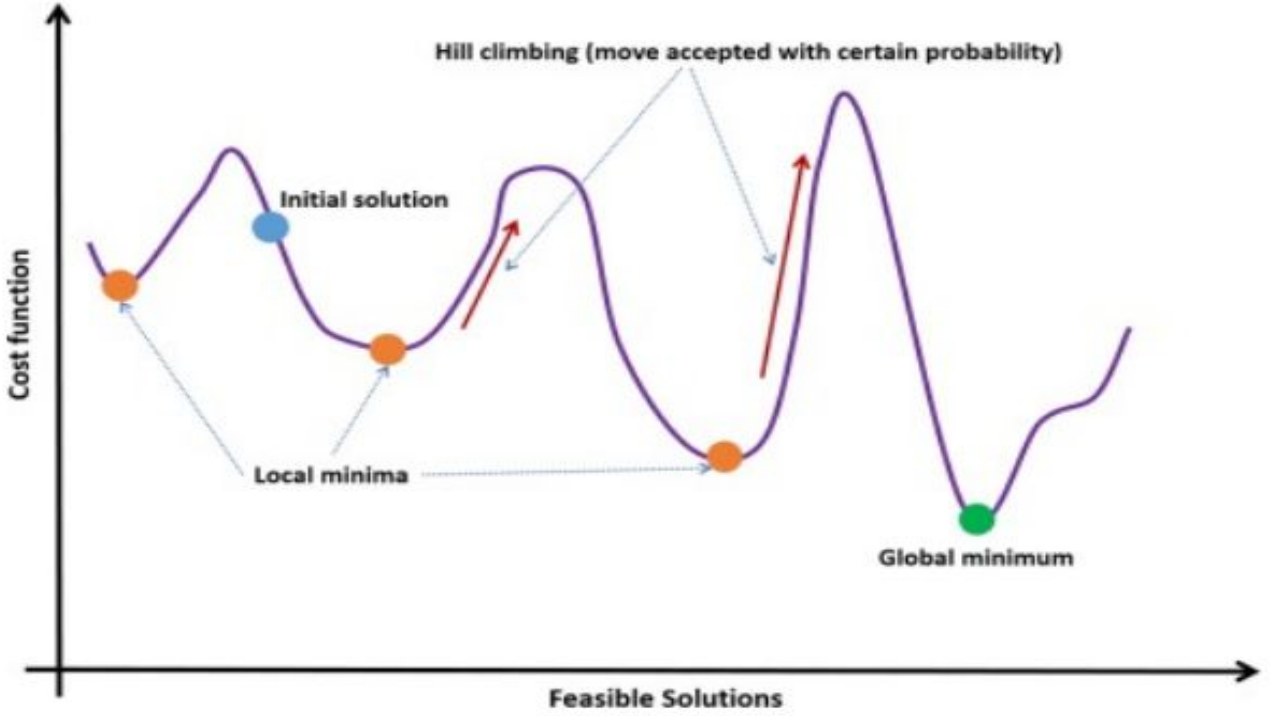
Source:[26][27]

Simulated Annealing

1. **Simulated annealing (SA) is a stochastic optimization algorithm used to find the global minimum of a cost function. It is a meta-heuristic in local search to solve the problem of getting stuck in local minima. The algorithm is based on the physical process of annealing in metallurgy, where metal is heated and then slowly cooled. By doing this, the metal is softened and prepared for further work. The algorithm was introduced in 1983 by Kirkpatrick et al. [93] and in 1985 by Černý.**
2. Its main idea is to find a global minimum of a specific objective function attempting to escape local minima in the search process. Due to low complexity, it can be used in many various optimization problems, not only related to the VRPs.
3. The concept includes that heating a solid material to a certain temperature at which it becomes liquid, followed by slow and controlled cooling down until the solid state is reached again and the metal particles are rearranged in a minimal energy molecular structure.

Source:[17][21]

Simulated Annealing



Source:[21]

Simulated Annealing: Pseudocode and functions used in a basic version of Simulated Annealing

Pseudocode	Explanation
$S^* = S_0$ $S = S_0$ $it = 0$ $T = T_0$	Input: Initialize the initial solution as best and as starting point, set iterations to 0, set T to the starting temperature.
while $it < N$ do	Termination criterion, can also be e.g., a time limit.
$S_{new} = generate\_neighbor(S)$	Select a neighbor of the current solution.
$delta = f(S_{new}) - f(S)$	Calculate the difference between new solution score and current solution score.
if $delta < 0$ then: $S = S_{new}$	Accept the new solution if it scores better than the current solution.
if $f(S) < f(S^*)$ then $S^* = S$	Update the best solution if the objective of the solution is better than the current optimal score.
else if $random() < e^{-\frac{delta}{T}}$ then $S = S_{new}$	If a random number is smaller than the acceptance value, also accept the new solution, even if it scores worse than the current one.
$it = it + 1$ $T = cooling(it)$	Add 1 to the number of iterations. Decrease the temperature.
return S^*	Output: the optimal solution.

Functions	Explanation
$generate\_neighbor(S)$	Generate a neighbor solution. Input: a solution
$f(S)$	Scoring function; lower is better in this example
$cooling(it)$	Adjust the temperature based on a schedule. Input: current iteration

Source:[17]

Simulated Annealing: How it works?

1. Initialization
   a. Begin with an initial Solution S0 and an initial temperature T0.
2. Neighborhood Search:
   a. At each step, a new solution S* is generated by making small change to the current solution.
3. Objective Function Evaluation:
   a. Here new solution is evaluated using object function. If S* provide better solution than previous solution S. It is accepted as the new solution.
4. Acception Probability:
   a. If S* new solution is worse than previous solution S, it still may accepted with a probability based on the temperature and the difference in objective function values.

score new solution score current solution

$$delta = f(S_{new}) - f(S)$$

$$acceptance\ value = e^{-\frac{delta}{temperature}}$$

Source:[17][28]

Simulated Annealing: How it works?Continued..

5. Acception Probability:

- **Cooling Schedule**: After each iteration, the temperature is decreased according to a predefined cooling schedule.
- **Termination**: Algorithm continues until the system reaches a low temperature meaning no more significant improvements are found.

Limitations

1. Parameter Sensitivity
2. Computational Time
3. Slow convergence

Source:[17][28]

Real world application of Simulated Annealing

Vehicle Routing Problems (VRP): Used for managing fleets to minimize costs under constraints like time windows (VRPTW), capacity, and delivery demands, as demonstrated by superior performance in Solomon benchmark sets.

Traveling Salesman Problem (TSP): SA is widely used to find the shortest route for a salesman visiting a set of cities, which is critical for courier services, logistics, and planning.

Hyperparameter Tuning: SA tunes parameters for machine learning models (e.g., neural network training) to avoid poor local minima, particularly in complex, non-convex landscapes.

Source:[14][16]

References

- [1]https://medium.com/@usb1508/navigating-the-complexity-understanding-p-np-np-complete-and-np-hard-problems-94137f9330f0, https://en.wikipedia.org/wiki/Computational_complexity_theory
- [2]https://en.wikipedia.org/wiki/P_(complexity), https://en.wikipedia.org/wiki/NP-completeness
- [3]https://jeffe.cs.illinois.edu/teaching/algorithms/book/12-nphard.pdf
- [4]https://en.wikipedia.org/wiki/Karp%27s_21_NP-complete_problems?utm_source=chatgpt.com
- [5]https://www.sciencedirect.com/topics/computer-science/approximation-algorithm#:~:text=In%20subject%20area:%20Computer%20Science,On%20this%20page
- [6]https://www.cs.cornell.edu/gomes/MYPAPERS/gomes-williams05.pdf
- [7]https://cs.stackexchange.com/questions/10182/difference-between-heuristic-and-approximation-algorithm
- [8]https://en.wikipedia.org/wiki/NP_(complexity)#:~:text=In%20computational%20complexity%20theory%2C%20NP,a%20solution%20to%20the%20problem.
- [9]https://www.usna.edu/Users/cs/wcbrown/courses/S18SI335/lec/l25/lec.html#:~:text=Here's%20what%20I%20think%20is,corresponding%20to%20getting%20a%200.
- [10]https://en.wikipedia.org/wiki/Nondeterministic_algorithm
- [11]https://www.sciencedirect.com/topics/computer-science/heuristic-algorithm

References

- [12]https://www.slideshare.net/slideshow/heuristic-search-in-artificial-intelligence-heuristic-function-in-ai-admissible-nonadmissible-digital-wave/251179169
- [13]http://theory.stanford.edu/~amitp/GameProgramming/Heuristics.html
- [14]https://en.wikipedia.org/wiki/Local_search_(optimization)
- [15]https://www.cs.cmu.edu/~15281-s23/coursenotes/localsearch/index.html
- [16]https://www.sciencedirect.com/topics/computer-science/local-search-algorithm
- [17]https://medium.com/p/87fd1e3676dd
- [18]https://www.datacamp.com/pt/tutorial/hill-climbing-algorithm-for-ai-in-python
- [19]https://en.wikipedia.org/wiki/Brute-force_search
- [20]https://www.geeksforgeeks.org/dsa/introduction-to-pattern-searching/
- [21]https://www.sciencedirect.com/topics/social-sciences/simulated-annealing
- [22]https://en.wikipedia.org/wiki/Greedy_randomized_adaptive_search_procedure
- [23]https://algorithmafternoon.com/stochastic/greedy_randomized_adaptive_search/
- [24]https://arxiv.org/html/2312.12663v1/

References

- [25]https://www.researchgate.net/publication/340267826_Greedy_randomized_adaptive_search_procedure_for_close_enough_orienteering_problem/figures
- [26]https://www.atlantis-press.com/journals/ijcis/25868746/view
- [27]https://energosteel.com/the-steel-tempering/
- [28]https://kindle-tech.com/faqs/what-are-the-different-methods-of-cooling-after-heat-treatment?srsltid=AfmBOorv4xrUWtDI0-uj5LtF1-QVKXce8bPqizwfYtnTz3SSv8F5RHZ5
- [29]https://www.geeksforgeeks.org/artificial-intelligence/what-is-simulated-annealing/
- [30]https://www.baeldung.com/cs/hill-climbing-algorithm
- [31]https://treeindev.net/article/algorithm-complexity-analysis
- [32]https://www.geeksforgeeks.org/dsa/difference-between-posteriori-and-priori-analysis/
- [33][9780262270830] Introduction to Algorithms, third edition
- [34]https://www.wscubetech.com/resources/dsa/space-complexity
- [35]https://medium.com/@noorfatimaafzalbutt/understanding-space-complexity-and-how-to-analyze-it-8a5c79895105
- [36]https://en.wikipedia.org/wiki/Asymptotic_analysis
- [37]https://www.cs.cornell.edu/courses/cs312/2004fa/lectures/lecture16.htm
- [38]https://www.wscubetech.com/resources/dsa/asymptotic-notation

References

- [39]https://www.geeksforgeeks.org/dsa/understanding-time-complexity-simple-examples/